

MODULE 1

Introduction to TinyML & ML Basics

TinyML — Condition Monitoring on Microcontrollers

Day 1 of 3

Day 1 roadmap

Module	Topic	Hands-on
1	TinyML intro + ML basics	Jupyter/Colab: pandas, MLP, MNIST, CNN
2	Embedded hardware: RAK4631 WisBlock	Blink, accelerometer, microphone bring-up
3	Data acquisition	Stream sensor data into Edge Impulse

3-day contents: **sensor** → **model** → **anomaly**

- **Day 1** — Foundations: ML basics, hardware, first data
- **Day 2** — The fan lab: build a dataset, engineer features, meet the frameworks
- **Day 3** — Train, deploy, detect anomalies, optimise

Theme throughout: condition monitoring of a rotating machine — a 120 mm PC fan standing in for the pumps you know.

**Typical power consumption
(Watt)**

100 - 1000's

ML on cloud
backend

10 - 100

ML on PC /
laptop

1 - 10

Edge ML

>1mW

Tiny ML

Typical HW Train Inference

GPU/TPU
cluster

✓

✓

4-16 core
computer

✓

✓

SBC/SOC

(✓)

✓

MCU
Or custom ASIC

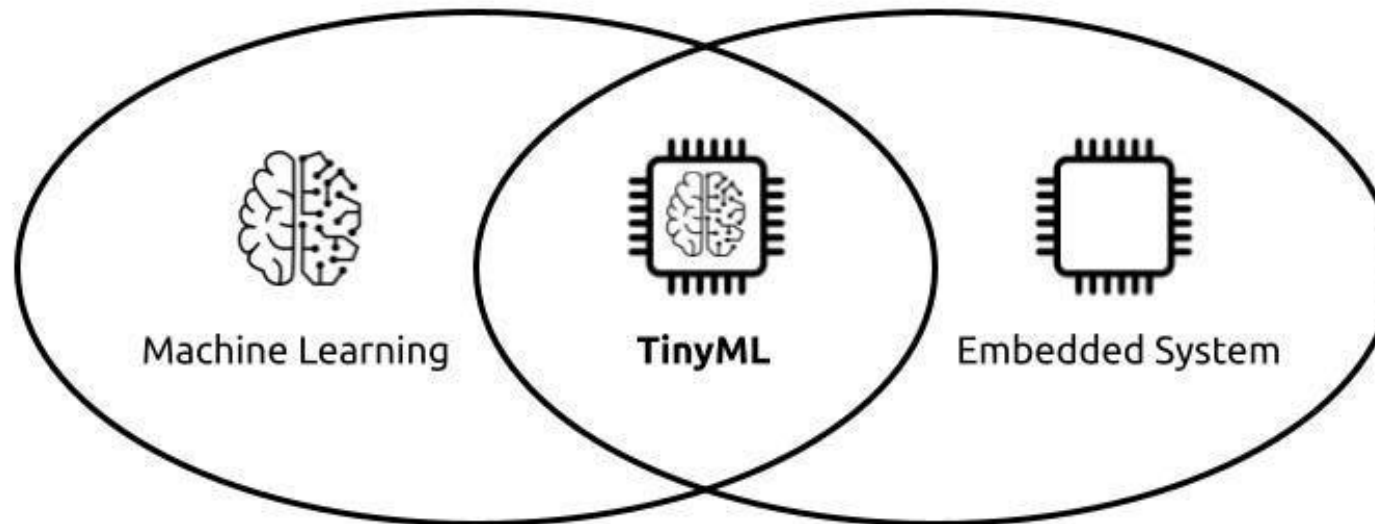
✗

✓

What is TinyML?

Machine learning inference **on microcontrollers**:

- kilobytes of RAM, megahertz clocks, milliwatts of power
- no OS (or a tiny RTOS), no GPU, often no network
- the model lives *next to the sensor*



Cloud ML: send data to the model. **TinyML: send the model to the data.**

Why ML *on* the microcontroller?

- **Latency** — decision in milliseconds, no round-trip
- **Power** — radio transmission costs far more energy than local compute
- **Privacy** — raw audio/vibration never leaves the device
- **Cost & bandwidth** — thousands of sensors × 16 kHz audio ≠ your network budget
- **Reliability** — works with no connectivity at all.

Energy cost of...

Here are some rough numbers for common components based on figures from [Smartphone Energy Consumption](#)

- ⌘ A display might use 400 milliwatts
- ⌘ Active cell radio might use 800 milliwatts
- ⌘ Bluetooth might use 100 milliwatts
- ⌘ Accelerometer is 21 milliwatts
- ⌘ Gyroscope is 130 milliwatts
- ⌘ GPS is 176 milliwatts.

Example MCU

Ambiq apollo4, Cortex M4F

Ultra-low supply current:

- 4 $\mu\text{A}/\text{MHz}$ active mode current
- Low-power sleep and deep sleep modes with selectable levels of RAM/cache retention

High-performance Arm Cortex-M4 Processor with FPU:

- 96/192 MHz operating modes
- Floating Point Unit, Memory Protection Unit
- Secure boot

Ultra-low-power memory:

- Up to 2 MB of non-volatile memory (NVM) for code/data
- Up to 2.75 MB of low power RAM for code/data

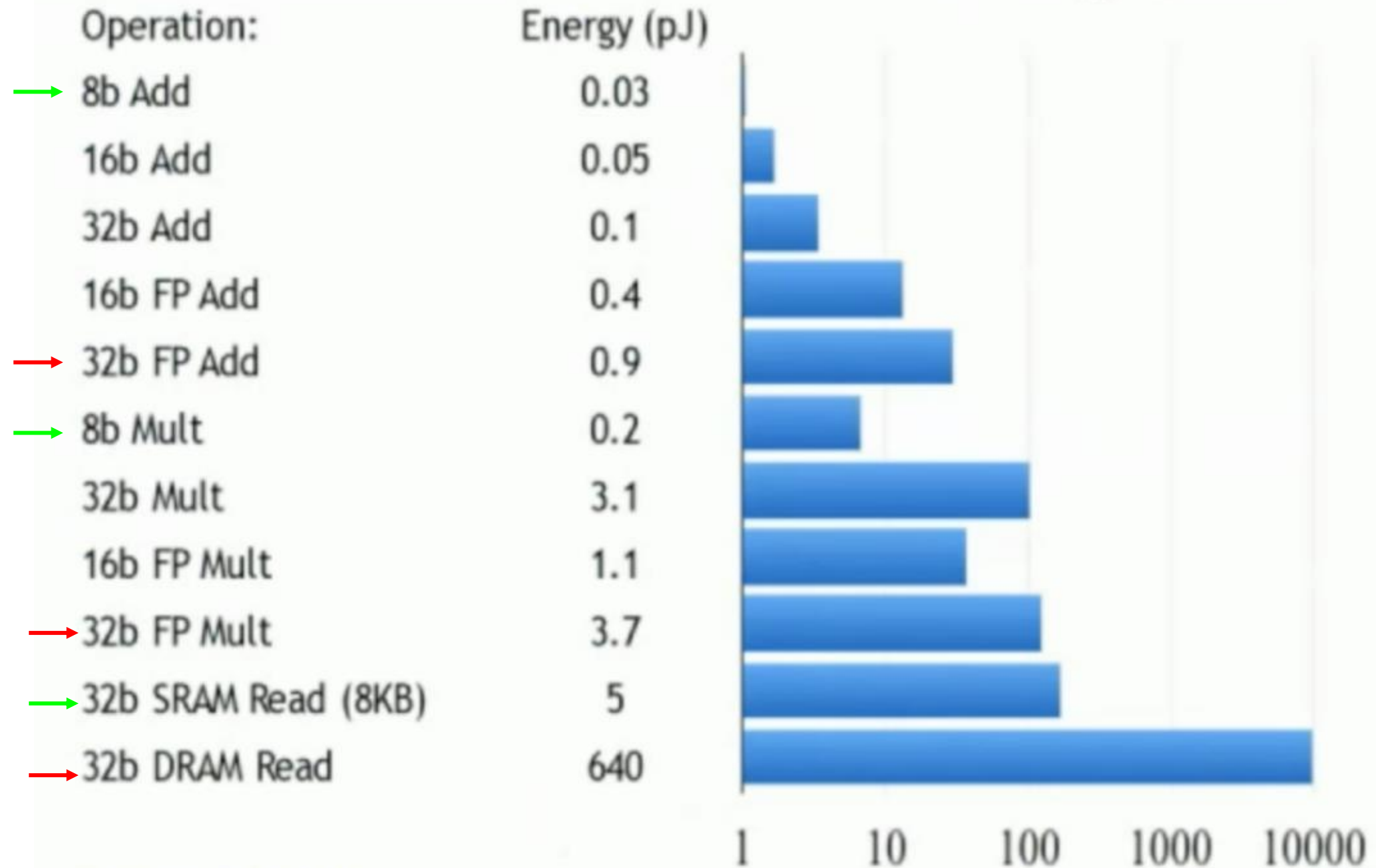
For single cycle instructions

@1MHz=1 μSec pr instruction

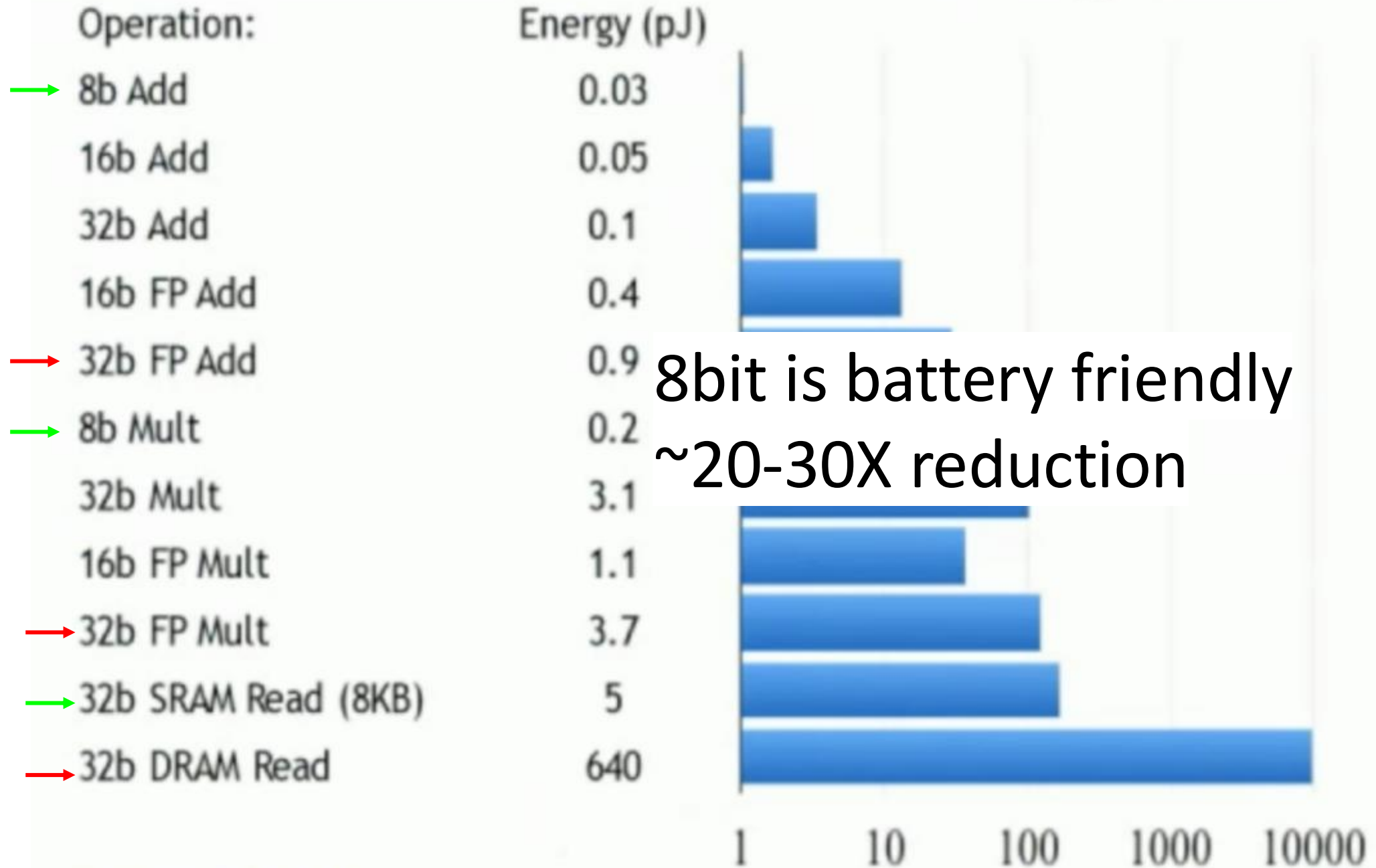
$4\mu\text{A} * 1,8\text{V} * 10^{-6}\text{S} \sim 7\text{pJ}$ per instruction



Relative Energy Cost



Relative Energy Cost



An Example...

For example, the MobileNetV2 image classification network takes 22 million ops in its smallest configuration.

If we know that a particular system takes 4 picojoules to execute a single op, then it will take $(4 \text{ picojoules} * 22,000,000) = \underline{88 \text{ microjoules of energy to execute}}$.

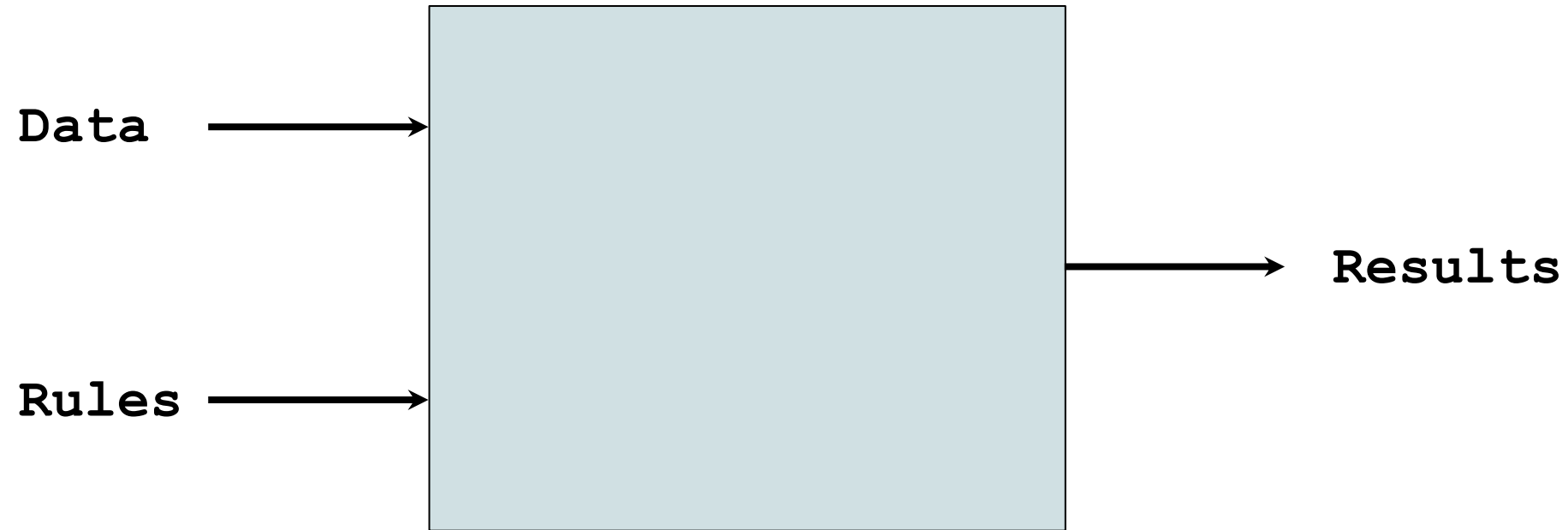
If we're analyzing one frame per second, that's only 88 microwatts, a 2032 coin battery (225mAh) could sustain continuously for nearly 4 months.

The TinyML pipeline

sensor → acquisition → dataset → features → train → convert → deploy → infer
(device) (device) (PC/cloud) (both!) (PC) (PC) (device) (device)

- **Train on PC, infer on device** — the rule
- Features must be computed *identically* in Python (training) and C (device)

Rule based / traditional programming



Data driven / ML programming



Machine learning in one slide

Instead of writing rules, we **learn them from examples**:

- Classical programming: `rules + data → answers`
- Machine learning: `data + answers → rules`

Supervised learning — we show the algorithm labelled examples:

"this vibration is `normal`", "this one is `imbalance`" — and it learns the mapping.

Classification vs regression

	Output	Example
Regression	a number	house price in \$1000s
Classification	a class + probabilities	<code>normal</code> / <code>imbalance</code> / <code>blocked</code>



A classifier **always** outputs probabilities summing to 1.0 — even on garbage input. (Day 3: anomaly detection fixes this.)

Train / validation / test split

- **Training set** — the model learns from this (~60–70%)
- **Validation set** — tune hyperparameters, detect overfitting during training
- **Test set** — touched **once**, at the end, for the honest number

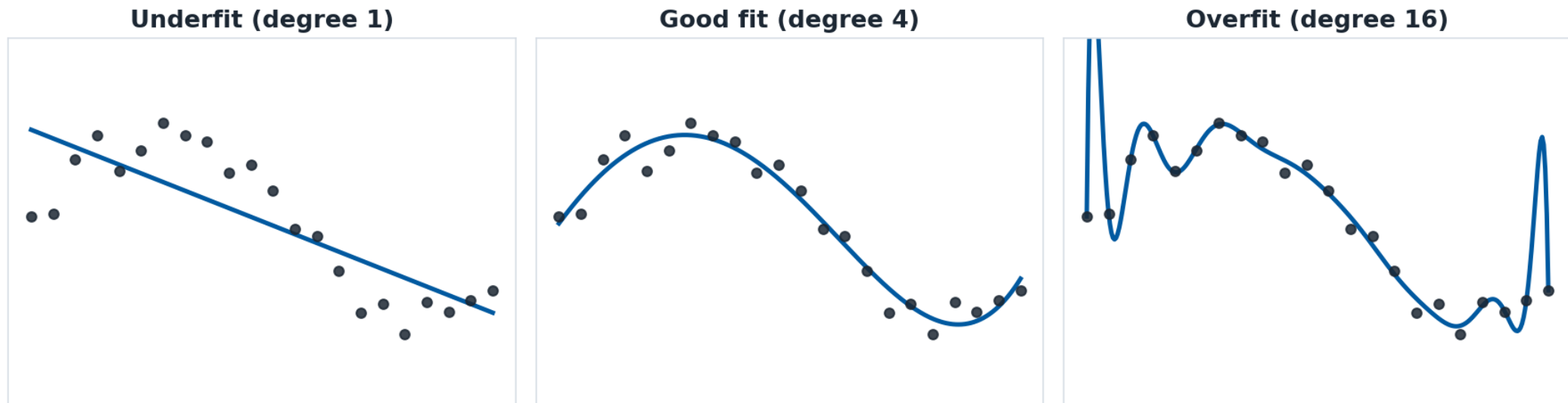
Cardinal sin: letting test data leak into training.

For sensor data: split by *recording session*, not by *sample*!

Overfitting

- **Underfit** — model too simple: bad on training AND test data
- **Good fit** — learns the pattern, generalises
- **Overfit** — memorises the training data: great training accuracy, poor test accuracy

Same data, three models — memorising $\neq$ learning



Watch the gap between training loss and validation loss.



Lab checkpoint 1 — pandas & house prices (~35 min)

[exercises/module1-ml-basics/](#) → parts 1–2

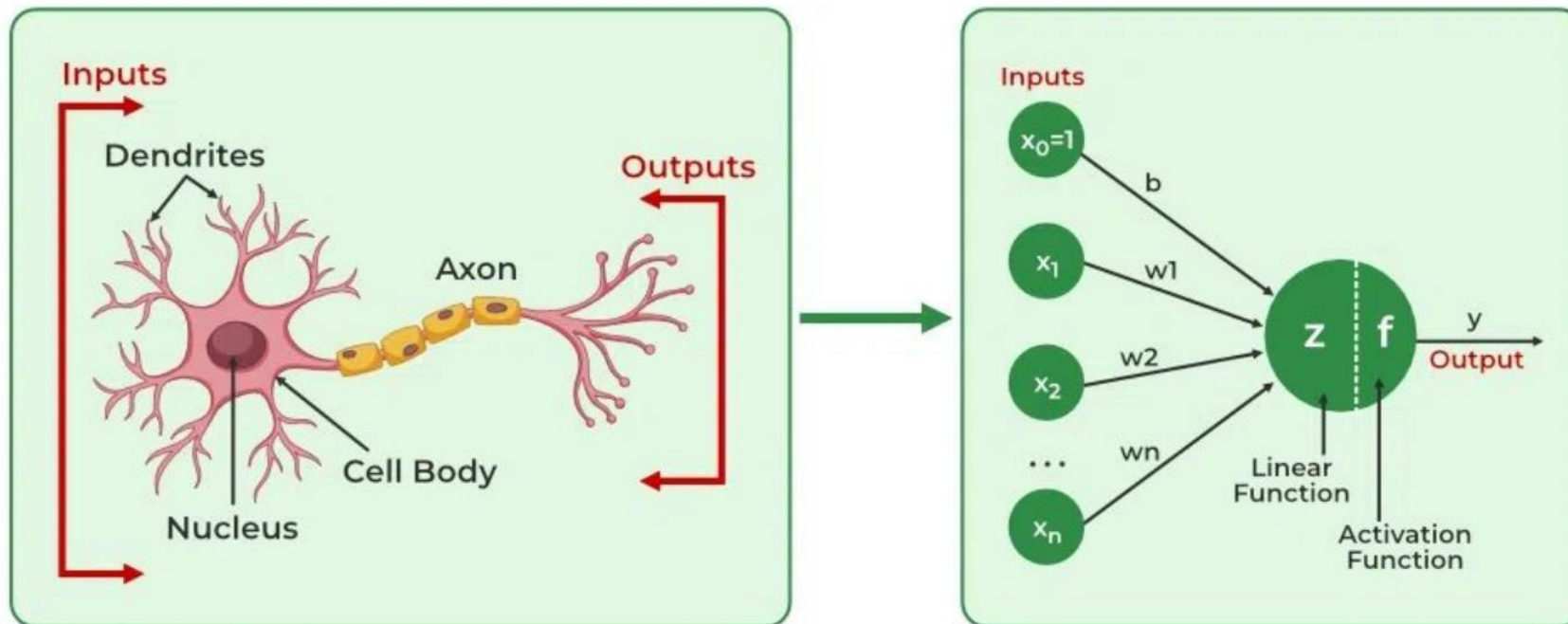
- Part 1: pandas warm-up (skimmable if you use it daily)
- Part 2: California-housing MLP — your first trained model
- **Done when:** you have a test-set error and can say whether it's any good

Part 2: Neural networks

the 20-minute tour

From biological to artificial neuron

Biological vs Artificial Neuron



Core terminology

Term	Meaning
Neuron	weighted sum + activation function
Weight (w)	learnable strength of a connection
Bias (b)	learnable offset — shifts the decision boundary
Activation $f(\cdot)$	nonlinearity: ReLU, sigmoid, tanh, softmax
Parameter	any trainable value (weights + biases) → model size
Feature	an input signal, or a learned representation inside the net

Layers

1. **Input layer** — receives raw data; its shape must match your data
 2. **Hidden layers** — the actual computation, weighted sums + activations
 3. **Output layer** — one neuron per regression target, or one per class
- "Deep" learning = more than one hidden layer. That's the whole secret.

Dense layers & the parameter count

Every neuron connects to **all** neurons in the previous layer.

```
params = inputs × outputs + outputs
```

Input	Output	Params	Size (f32)
12	32	416	1.6 KB
128	64	8,256	32 KB
784 (28×28)	128	100,480	393 KB



784→128 alone claims **~40 % of our 1 MB flash** — weights live in *flash*; RAM (256 KB) is reserved for activations and buffers. This is why CNNs exist.

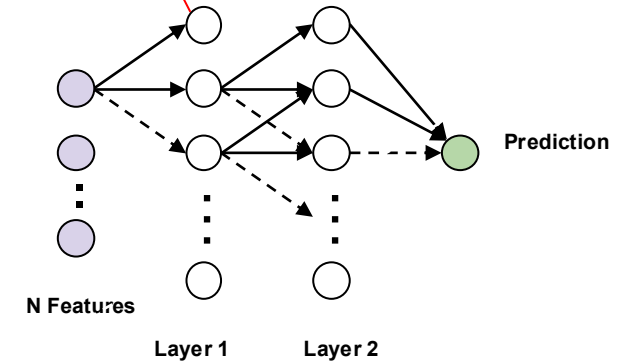
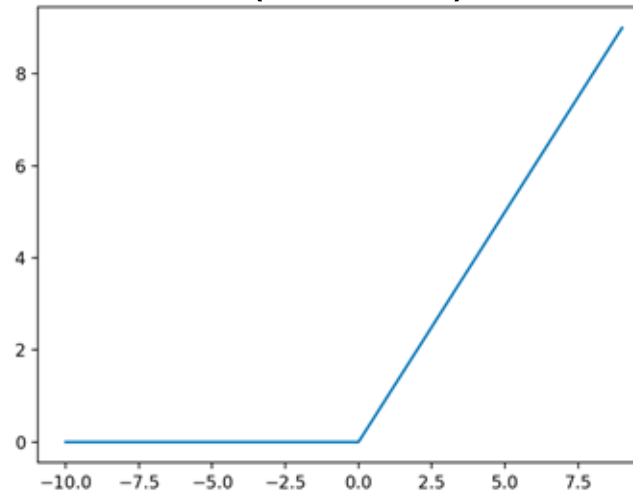
Deep Learning = More layers

All nodes connects to all layer by layer...

Each neuron computes a weighted sum :

$$\text{output} = a(z) = a \left(\left(\sum_{i=1}^n x_i \times w_i \right) + b \right)$$

ReLu (rectifier)



Activation functions, adding non-linearities

Activation function sit between layers in Neural networks.

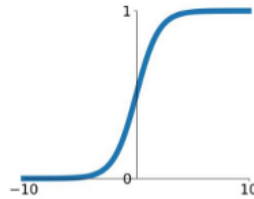
Their non-linearity allows the network to learn complex, non-linear relationships in data.

Without non-linearity additional layers would only collapse into a single linear transform.

Activation Functions

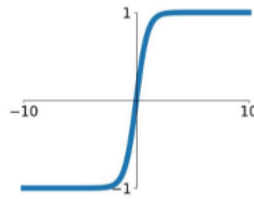
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



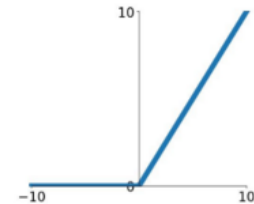
tanh

$$\tanh(x)$$



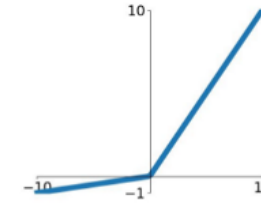
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

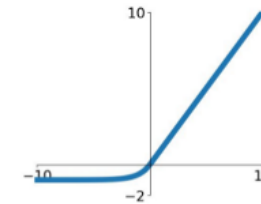


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation functions — the MCU view

Function	Use case	MCU cost
ReLU <code>max(0,x)</code>	hidden layers (default)	very low
Leaky ReLU	hidden layers (alt)	very low
Sigmoid	binary output	high (exp)
Tanh	hidden, centered	high (exp)
Softmax	multi-class output	moderate

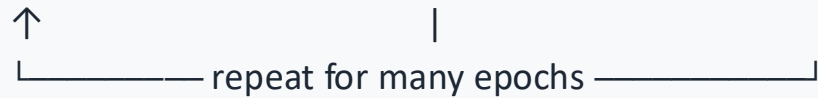
On microcontrollers: ReLU is the default.

Softmax turns the output layer's raw scores into probabilities:

$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$ — always positive, always sums to **1.0**.

How the network learns

forward pass → compute loss → backpropagation → update weights



- **Loss function** — how wrong are we? (MSE for regression, cross-entropy for classification)
- **Backpropagation** — chain rule assigns blame to every weight
- **Optimizer** (SGD, Adam) — nudges loss downhill: $w \leftarrow w - \eta \cdot \nabla L$

$$w \leftarrow w - \eta \cdot \nabla L$$

Epochs, batches, learning rate

- **Epoch** — one full pass through the training set
- **Batch size** — samples processed before each weight update
- **Learning rate η** — step size. *The* most critical hyperparameter:
 - too high → training diverges, loss explodes
 - too low → training crawls, or gets stuck

You will break this on purpose in the MNIST exercise.

A Keras model in 8 lines

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Input(shape=(13,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1)          # regression: 1 linear output
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.fit(X_train, y_train, epochs=100, validation_split=0.2)
```

Today's three datasets

Dataset	Task	Input	Why
California housing	regression	8 numeric features	hand-made features
MNIST digits	10-class classification	28×28 grayscale	raw pixels, dense NN
CIFAR-10	10-class classification	32×32 RGB	CNN teaser

Features vs raw data — the running theme

- **Housing:** features engineered by humans → tiny model works
- **MNIST:** raw pixels → the network learns its own features → more params

Typical in TinyML : good hand-crafted features (RMS, FFT peaks...)

buy you a **much smaller model**. Day 2 is dedicated to this.



Lab checkpoint 2 — MNIST & CIFAR (~50 min)

[exercises/module1-ml-basics/](#) → parts 3–4

(parts 1–2 were checkpoint 1, before the NN section)

3. **MNIST** — dense NN classifier, play with learning rate

4. **CIFAR-10** — CNN teaser (optional if time is short)

All run in **Google Colab** (zero install) or local Jupyter.

Discussion checkpoints in the README.

Module 1 wrap-up

You now have:

- ✓
 - The TinyML pipeline in your head
- ✓
 - Supervised learning, splits, overfitting
- ✓
 - NN anatomy: neurons, layers, activations, training loop
- ✓
 - Hands-on Keras: regression + classification

Next: the hardware. 256 KB of RAM, sensors and and microphone.

Sources & acknowledgements

This course builds on excellent open material:

- **TinyML on edX** (HarvardX) — Prof. **Vijay Janapa Reddi** et al.
Figures and framing reused **with permission** (attributed per slide)
- **A. Géron**, *Hands-On Machine Learning* notebooks ([hands-on-ml3](#) , Apache-2.0)
- **Jon Nordby** — **emlearn** (open-source ML inference for MCUs)
- **Fraunhofer IMS** — **AlfES** (AI framework for embedded systems)

Thank you to all authors for making high-quality TinyML material open.